



**NANYANG
TECHNOLOGICAL
UNIVERSITY**

Data Structures & Algorithms in Python

Linked Lists – Lab Answers

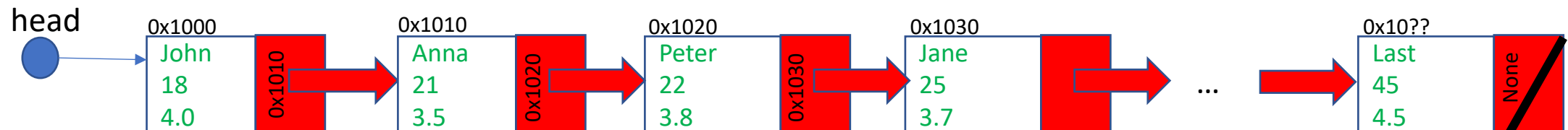
Dr. Owen Noel Newton Fernando

College of Engineering

School of Computer Science and Engineering

linked lists Recap

- A **LinkedList** consists of a connected sequence of **Link/Node** objects
- Each **node** contains a reference to a datum and a reference to the next **node** in the sequence.
 - The datum is just an object. It could be just an int or a string, or it could contain a key/data pair.
- The last node in the sequence refers to **None** as its next **node**
- The sequence of **nodes** is accessed via a “header” object that contains a reference to the first **node** in the sequence.



Q01

1. Write a function `moveOddItemstoback()` that moves all odd integers to the back of a linked list.

The function prototype

```
def moveOddItemstoback(head) :
```

Q01 - Answer

Let's go through the step-by-step

1. Class Definitions

```
class Node:  
    def __init__(self, data):  
        self.data = data  
        self.next = None
```

- **Node** represents a single element of the linked list.
- data holds the value, next points to the next node (or None if it's the last node).

Q01 - Answer

1. Class Definitions

```
class LinkedList:  
    def __init__(self):  
        self.head = None  
        self.size = 0
```

class LinkedList:

This starts the definition of a new class called LinkedList

Q01 - Answer

2. Functions

Finding a node by index

Checks if the index is **out of bounds**.

For example:

If size = 5, valid indexes are 0 to 4.

Negative indexes are also not allowed here.

If invalid → raise an error immediately.



```
def findNode(self, index):
```

```
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")
```

```
    if self.head is None:  
        raise ValueError("List is empty")
```

```
    cur = self.head  
    while index > 0:  
        cur = cur.next  
        index -= 1  
    return cur
```

Q01 - Answer

2. Functions

Finding a node by index

- If there's no head node, the list is empty.
- There's no point searching, so it raises an error.



```
def findNode(self, index):  
  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")  
    if self.head is None:  
        raise ValueError("List is empty")  
  
    cur = self.head  
    while index > 0:  
        cur = cur.next  
        index -= 1  
    return cur
```

Q01 - Answer

2. Functions

Finding a node by index

```
def findNode(self, index):  
  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")  
    if self.head is None:  
        raise ValueError("List is empty")  
  
    cur = self.head  
    while index > 0:  
        cur = cur.next  
        index -= 1  
    return cur
```

- Starts traversal from the first node in the list.



Q01 - Answer

2. Functions

- **Finding a node by index**

- Moves forward through the linked list index times.
- Each loop moves cur to the next node and decrements index.
- When index reaches 0, cur is pointing to the desired node.



```
def findNode(self, index):  
  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")  
    if self.head is None:  
        raise ValueError("List is empty")  
  
    cur = self.head  
    while index > 0:  
        cur = cur.next  
        index -= 1  
    return cur
```

Q01 - Answer

2. Functions

- Finding a node by index

```
def findNode(self, index):  
  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")  
    if self.head is None:  
        raise ValueError("List is empty")  
  
    cur = self.head  
    while index > 0:  
        cur = cur.next  
        index -= 1  
    return cur
```

- Returns the node object at the requested position.

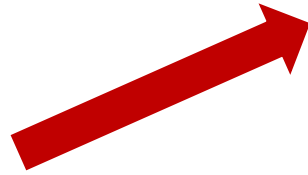


Q01 - Answer

2. Functions

- **Inserting a node**

- Validates that the insertion position is allowed.
- You can insert at self.size (end of list), but not beyond.
- Example: If size = 3, valid indexes are 0, 1, 2, 3.



```
def insertNode(self, data, index):  
    if index < 0 or index > self.size:  
        raise ValueError("Invalid position")  
  
    new_node = Node(data)  
  
    if index == 0:  
        new_node.next = self.head  
        self.head = new_node  
        self.size += 1  
        return True  
  
    prev_node = self.findNode(index - 1)  
    if prev_node is not None:  
        new_node.next = prev_node.next  
        prev_node.next = new_node  
        self.size += 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

- **Inserting a node**

- Creates a new node with the given data.
- Initially, new_node.next is None.



```
def insertNode(self, data, index):  
    if index < 0 or index > self.size:  
        raise ValueError("Invalid position")  
  
    new_node = Node(data)  
  
    if index == 0:  
        new_node.next = self.head  
        self.head = new_node  
        self.size += 1  
        return True  
  
    prev_node = self.findNode(index - 1)  
    if prev_node is not None:  
        new_node.next = prev_node.next  
        prev_node.next = new_node  
        self.size += 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

- **Inserting a node**

- Special case: inserting at the start.
- The new node's next is set to the current head.
- The head is updated to be the new node.
- Increases the list size by 1.
- Returns True to indicate success.



```
def insertNode(self, data, index):  
    if index < 0 or index > self.size:  
        raise ValueError("Invalid position")
```

```
    new_node = Node(data)
```

```
    if index == 0:  
        new_node.next = self.head  
        self.head = new_node  
        self.size += 1  
        return True
```

```
    prev_node = self.findNode(index - 1)  
    if prev_node is not None:  
        new_node.next = prev_node.next  
        prev_node.next = new_node  
        self.size += 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

- Inserting a node

- For any position other than 0, you first need to find the node before the insertion point.
- Uses your **findNode()** method to get it.
- Example: inserting at index 3 → you first find index 2.

```
def insertNode(self, data, index):  
    if index < 0 or index > self.size:  
        raise ValueError("Invalid position")  
  
    new_node = Node(data)  
  
    if index == 0:  
        new_node.next = self.head  
        self.head = new_node  
        self.size += 1  
        return True  
  
    prev_node = self.findNode(index - 1)  
    if prev_node is not None:  
        new_node.next = prev_node.next  
        prev_node.next = new_node  
        self.size += 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

- Inserting a node

- Links the new node into the list:
 - **new_node.next** points to what **prev_node** used to point to.
 - **prev_node.next** is updated to point to the new node.
- Increments size.
- Returns True if succeeds, else false.

```
def insertNode(self, data, index):  
    if index < 0 or index > self.size:  
        raise ValueError("Invalid position")
```

```
    new_node = Node(data)
```

```
    if index == 0:  
        new_node.next = self.head  
        self.head = new_node  
        self.size += 1  
        return True
```

```
    prev_node = self.findNode(index - 1)  
    if prev_node is not None:  
        new_node.next = prev_node.next  
        prev_node.next = new_node  
        self.size += 1  
        return True  
    return False
```

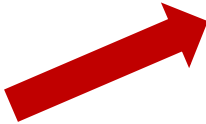


Q01 - Answer

2. Functions

- **Removing a node**

- Makes sure the index is within valid bounds ($0 \leq \text{index} < \text{size of the list}$).
- If it's not, a ValueError is raised.



```
def removeNode(self, index):  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")  
  
    if self.head is None:  
        return False  
  
    if index == 0:  
        cur = self.head  
        self.head = cur.next  
        self.size -= 1  
        return True  
  
    pre = self.findNode(index - 1)  
    if pre is not None and pre.next is not None:  
        cur = pre.next  
        pre.next = cur.next  
        self.size -= 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

- Removing a node

- If the list has no nodes, removal is impossible, so it returns False.



```
def removeNode(self, index):  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")  
  
    if self.head is None:  
        return False  
  
    if index == 0:  
        cur = self.head  
        self.head = cur.next  
        self.size -= 1  
        return True  
  
    pre = self.findNode(index - 1)  
    if pre is not None and pre.next is not None:  
        cur = pre.next  
        pre.next = cur.next  
        self.size -= 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

- **Removing a node**

- If the first node (**index == 0**) is being removed:
 - Store the current head in cur.
 - Update the head to the second node (cur.next).
 - Reduce the size by 1.
 - Return True indicating success.



```
def removeNode(self, index):  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")  
  
    if self.head is None:  
        return False  
  
    if index == 0:  
        cur = self.head  
        self.head = cur.next  
        self.size -= 1  
        return True  
  
    pre = self.findNode(index - 1)  
    if pre is not None and pre.next is not None:  
        cur = pre.next  
        pre.next = cur.next  
        self.size -= 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

• Removing a node

- First, find the previous node (**index - 1**) using `findNode()`.
- Make sure `pre` exists and that there's a node after it (**`pre.next`**).
- Store the node to be deleted in `cur`.
- Link the previous node directly to the node after the one being deleted (**`pre.next = cur.next`**), skipping `cur`.
- Reduce the size by 1.
- Return `True`.



```
def removeNode(self, index):  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")
```

```
    if self.head is None:  
        return False
```

```
    if index == 0:  
        cur = self.head  
        self.head = cur.next  
        self.size -= 1  
        return True
```

```
    pre = self.findNode(index - 1)  
    if pre is not None and pre.next is not None:  
        cur = pre.next  
        pre.next = cur.next  
        self.size -= 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

- printing the entire linked list

- **cur** starts at the head of the list.



```
def printList(self):  
    cur = self.head  
    if cur is None:  
        print("Empty")  
        return  
  
    while cur is not None:  
        print(cur.data, end=" -> ")  
        cur = cur.next  
    print("None")
```

Q01 - Answer

2. Functions

- **printing the entire linked list**

- If the head is None, the list is empty.
- Print "Empty" and return immediately since there is nothing to print.



```
def printList(self):  
    cur = self.head  
    if cur is None:  
        print("Empty")  
        return  
  
    while cur is not None:  
        print(cur.data, end=" -> ")  
        cur = cur.next  
    print("None")
```

Q01 - Answer

2. Functions

- **printing the entire linked list**

- Loop through the list starting from the head.
- For each node, print its data followed by " -> " without moving to a new line (end=" -> ").
- Move cur to the next node.



```
def printList(self):  
    cur = self.head  
    if cur is None:  
        print("Empty")  
        return
```

```
    while cur is not None:  
        print(cur.data, end=" -> ")  
        cur = cur.next  
    print("None")
```

Q01 - Answer

2. Functions

- printing the entire linked list

- After the loop finishes (i.e., when cur becomes None), print "None" to indicate the end of the list.



```
def printList(self):  
    cur = self.head  
    if cur is None:  
        print("Empty")  
        return  
  
    while cur is not None:  
        print(cur.data, end=" -> ")  
        cur = cur.next  
    print("None")
```

Q01 - Answer

2. Functions

Now we will look at the `moveOdditemstoback()` Function

- If the list is empty (head is None) or has only one node, there's nothing to rearrange, so we return it as-is.



```
def moveOdditemstoback(head):  
    if not head or not head.next:  
        return head
```

```
    even_head = None  
    even_tail = None  
    odd_head = None  
    odd_tail = None  
    current = head
```

#Continued...

Q01 - Answer

2. Functions

Now we will look at the `moveOdditemstoback()` Function

- We create **two lists** internally:
 - **even_head & even_tail** → start and end of the even-number list.
 - **odd_head & odd_tail** → start and end of the odd-number list.



```
def moveOdditemstoback(head):  
    if not head or not head.next:  
        return head
```

```
even_head = None  
even_tail = None  
odd_head = None  
odd_tail = None  
current = head
```

#Continued...

Q01 - Answer

2. Functions

Now we will look at the moveOdditemstoback() Function

- This means keep going until current becomes None (i.e., end of the linked list).
- At the start, current = head.



```
while current:  
    next_node = current.next  
    current.next = None
```

#Continued...

Q01 - Answer

2. Functions

Now we will look at the moveOdditemstoback() Function

- We temporarily store the next node so we can continue the loop later.
- This is important because in the next step we're going to cut the link from current.



```
while current:  
    next_node = current.next  
    current.next = None
```

#Continued...

Q01 - Answer

2. Functions

Now we will look at the moveOdditemstoback() Function

- We check if the data in the node is divisible by 2.
- If yes → node is even → goes into the even list.



```
if current.data % 2 == 0:
    if not even_head:
        even_head = current
        even_tail = current
    else:
        even_tail.next = current
        even_tail = current
else:
    if not odd_head:
        odd_head = current
        odd_tail = current
    else:
        odd_tail.next = current
        odd_tail = current
```

current = next_node

#Continued...

Q01 - Answer

2. Functions

Now we will look at the moveOdditemstoback() Function

- If **even_head** is **None** → this is the first even node → set both **head** and **tail** to **current**.
- Otherwise:
 - Attach this node after the tail:
even_tail.next = current
 - Move the tail pointer to the new node:
even_tail = current.



```
if current.data % 2 == 0:
    if not even_head:
        even_head = current
        even_tail = current
    else:
        even_tail.next = current
        even_tail = current
else:
    if not odd_head:
        odd_head = current
        odd_tail = current
    else:
        odd_tail.next = current
        odd_tail = current
```

current = next_node

#Continued...

Q01 - Answer

2. Functions

Now we will look at the `moveOdditemstoback()` Function

- If not **even**, it must be **odd**.
- Same logic as the even list:
 - If **odd_head** is **None** → first odd node → set **head** and **tail** to **current**.
 - Otherwise, attach to the odd tail and move the tail pointer.



```
if current.data % 2 == 0:
    if not even_head:
        even_head = current
        even_tail = current
    else:
        even_tail.next = current
        even_tail = current
```

```
else:
    if not odd_head:
        odd_head = current
        odd_tail = current
    else:
        odd_tail.next = current
        odd_tail = current
```

`current = next_node`

`#Continued...`

Q01 - Answer

2. Functions

Now we will look at the `moveOdditemstoback()` Function

- We jump to the next node we saved earlier.
- Loop continues until current becomes None.

```
if current.data % 2 == 0:
    if not even_head:
        even_head = current
        even_tail = current
    else:
        even_tail.next = current
        even_tail = current
else:
    if not odd_head:
        odd_head = current
        odd_tail = current
    else:
        odd_tail.next = current
        odd_tail = current
```

`current = next_node`

#Continued...

Q01 - Answer

2. Functions

Now we will look at the `moveOdditemstoback()` Function

- Merge even and odd lists
- If one group is empty, just return the other.
- Otherwise, connect the even list's tail to the odd list's head.



```
if not even_head:
    return odd_head # No evens, only odds

if not odd_head:
    return even_head # No odds, only evens

even_tail.next = odd_head # Connect even list
to odd list

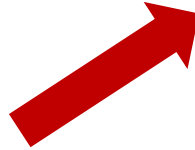
odd_tail.next = None # Ensure list ends
properly

return even_head
```

Q01 - Answer

Testing our function

- Creates an instance of **LinkedList** (our custom class).
- At this point: **linked_list.head = None** and **linked_list.size = 0**.
- Then, Prompt the user for input.



```
if __name__ == "__main__":
    linked_list = LinkedList()

    print("Enter a list of numbers, terminated by any non-
digit character: ", end="")
    input_string = input()
    numbers = input_string.split()

    counter = 0
    for num in numbers:
        try:
            linked_list.insertNode(int(num), counter)
            counter += 1
        except ValueError:
            break

    print("\nBefore:", end=" ")
    linked_list.printList()
    linked_list.head = moveOdditemstoback(linked_list.head)
    print("After:", end=" ")
    linked_list.printList()
```

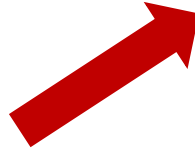
Q01 - Answer

Testing our function

- Split the input into separate items
- Splits the string into a list of substrings based on whitespace.

Example:

- "4 7 8 3 2 q".split() → ['4', '7', '8', '3', '2', 'q']



```
if __name__ == "__main__":
    linked_list = LinkedList()

    print("Enter a list of numbers, terminated by any non-
digit character: ", end="")
    input_string = input()
    numbers = input_string.split()

    counter = 0
    for num in numbers:
        try:
            linked_list.insertNode(int(num), counter)
            counter += 1
        except ValueError:
            break

    print("\nBefore:", end=" ")
    linked_list.printList()
    linked_list.head = moveOdditemstoback(linked_list.head)
    print("After:", end=" ")
    linked_list.printList()
```

Q01 - Answer

Testing our function

- Loops over each element in numbers.
- Tries to convert it to an integer with `int(num)`:
- If it's a digit, it calls **insertNode** to put it at position **counter**.
- counter ensures elements are inserted in the right order (like appending).
- If `int(num)` fails (e.g., 'q'), it raises **ValueError** and breaks out of the loop.
- So entering "4 7 8 3 2 q" will only insert 4 → 7 → 8 → 3 → 2 into the linked list.



```
if __name__ == "__main__":
    linked_list = LinkedList()

    print("Enter a list of numbers, terminated by any non-
digit character: ", end="")
    input_string = input()
    numbers = input_string.split()

    counter = 0
    for num in numbers:
        try:
            linked_list.insertNode(int(num), counter)
            counter += 1
        except ValueError:
            break

    print("\nBefore:", end=" ")
    linked_list.printList()
    linked_list.head = moveOdditemstoback(linked_list.head)
    print("After:", end=" ")
    linked_list.printList()
```

Q01 - Answer

Testing our function

- Print the linked list before modification
- Calls your printList() method to show the current list.



```
if __name__ == "__main__":
    linked_list = LinkedList()

    print("Enter a list of numbers, terminated by any non-
digit character: ", end="")
    input_string = input()
    numbers = input_string.split()

    counter = 0
    for num in numbers:
        try:
            linked_list.insertNode(int(num), counter)
            counter += 1
        except ValueError:
            break

    print("\nBefore:", end=" ")
    linked_list.printList()
    linked_list.head = moveOdditemstoback(linked_list.head)
    print("After:", end=" ")
    linked_list.printList()
```

Q01 - Answer

Testing our function

- Rearrange the list
- Passes the head of the list to **moveOdditemstoback** (the function we wrote earlier).
- That function rearranges the nodes so even numbers come first, odd numbers go last.
- The returned new head of the list is stored back in **linked_list.head**.

```
if __name__ == "__main__":
    linked_list = LinkedList()

    print("Enter a list of numbers, terminated by any non-
digit character: ", end="")
    input_string = input()
    numbers = input_string.split()

    counter = 0
    for num in numbers:
        try:
            linked_list.insertNode(int(num), counter)
            counter += 1
        except ValueError:
            break

    print("\nBefore:", end=" ")
    linked_list.printList()
    linked_list.head = moveOdditemstoback(linked_list.head)
    print("After:", end=" ")
    linked_list.printList()
```



Q01 - Answer

Testing our function

- Print the modified list
- Shows the rearranged linked list after moving all odd items to the end.

```
if __name__ == "__main__":  
    linked_list = LinkedList()  
  
    print("Enter a list of numbers, terminated by any non-  
digit character: ", end="")  
    input_string = input()  
    numbers = input_string.split()  
  
    counter = 0  
    for num in numbers:  
        try:  
            linked_list.insertNode(int(num), counter)  
            counter += 1  
        except ValueError:  
            break  
  
    print("\nBefore:", end=" ")  
    linked_list.printList()  
    linked_list.head = moveOdditemstoback(linked_list.head)  
    print("After:", end=" ")  
    linked_list.printList()
```



2. Write the function `removeNode()` using the `LinkedList` structure defined in the lecture materials. The function should remove a node at the specified index from the linked list.

The function definition is as follows:

```
def removeNode(ll, index):
```

Q02 - Answer

The difference between our earlier removeNode() method in Q01 and the one we are to implement in this question

Earlier version	This new version
Implemented as a method of the LinkedList class (e.g., ll.removeNode(index))	Implemented as a standalone function (removeNode(ll, index))
Called like: linked_list.removeNode(2)	Called like: removeNode(linked_list, 2)
Has direct access to self (the linked list itself)	Receives ll (the linked list) as a parameter
Otherwise, the logic is nearly identical	Logic is duplicated but adapted for a function

Q02 - Answer

Now we will go through the new removeNode() Function

- ll.head points to the first node of the list.
- If ll.head is None, the list has no nodes → nothing to remove.
- Prints a message and returns False to indicate removal failed.



The standalone function as specified in the question

```
def removeNode(ll, index):
```

```
    if ll.head is None:  
        print("List is empty")  
        return False
```

Handle removing first node (index 0)

```
if index == 0:  
    cur = ll.head  
    ll.head = cur.next  
    del cur  
    ll.size -= 1  
    return True
```

Find node before removal point

```
current = ll.head  
count = 0  
while current and count < index - 1:  
    current = current.next  
    count += 1
```

#Continued...

Q02 - Answer

Now we will go through the new removeNode() Function

- Removing the First Node (Special Case)

Why special case?

- The first node is different because removing it means **updating the head pointer**.
- **cur = ll.head** → store reference to the first node.
- **ll.head = cur.next** → move head to the second node.
- **del cur** → delete the old first node from memory.
- Decrement the size of the list.
- Return True → removal was successful.



```
# The standalone function as specified in the question
def removeNode(ll, index):
    if ll.head is None:
        print("List is empty")
        return False
```

```
# Handle removing first node (index 0)
```

```
if index == 0:
    cur = ll.head
    ll.head = cur.next
    del cur
    ll.size -= 1
    return True
```

```
# Find node before removal point
```

```
current = ll.head
count = 0
while current and count < index - 1:
    current = current.next
    count += 1
```

#Continued...

Q02 - Answer

Now we will go through the new removeNode() Function

- Start from the head.
- Loop until:
 - You've reached the node before the one to delete (**index - 1**), or
 - The list ends (**current** is **None**).
- Example: If removing node at index 3, you stop at index 2.



```
# The standalone function as specified in the question
def removeNode(ll, index):
    if ll.head is None:
        print("List is empty")
        return False
```

```
# Handle removing first node (index 0)
if index == 0:
    cur = ll.head
    ll.head = cur.next
    del cur
    ll.size -= 1
    return True
```

```
# Find node before removal point
current = ll.head
count = 0
while current and count < index - 1:
    current = current.next
    count += 1
```

#Continued...

Q02 - Answer

Now we will go through the new removeNode() Function

- Index Validity Check
- If **current** is **None** → index was too big.
- If **current.next** is **None** → there's no node to remove at that index.
- Return False to indicate failure.



```
# Check if position is valid
if not current or not current.next:
    print("Index out of range")
    return False
```

```
# Remove the node
temp = current.next
current.next = temp.next
del temp
ll.size -= 1
return True
```

#Continued...

Q02 - Answer

Now we will go through the new removeNode() Function

- **Removing the Target Node**
- **temp = current.next** → store reference to the node we want to remove.
- **current.next = temp.next** → skip over the node to delete, linking directly to the next one.
- **del temp** → free memory for that node. Decrement size.
- Return True for success.



```
# Check if position is valid
if not current or not current.next:
    print("Index out of range")
    return False
```

```
# Remove the node
temp = current.next
current.next = temp.next
del temp
ll.size -= 1
return True
```

3. Write a function `split()` that copies the contents of a linked list into two other linked lists.

The function prototype is given below:

`def split(ll):`

The function should copy nodes with even indices (0, 2, 4, etc.) to `evenList` and nodes with odd indices (1, 3, 5, etc.) to `oddList`. The original linked list should remain unmodified.

Q03 - Answer

Now we will go through the split() Function

- Handle Empty List
- If **ll.head** is **None** → list is empty.
- Returns two empty **LinkedList** objects.
- This prevents errors when trying to loop over an empty list.



```
def split(ll):  
    if not ll.head:  
        return LinkedList(), LinkedList()  
  
    even_list = LinkedList()  
    odd_list = LinkedList()  
    current = ll.head  
    index = 0  
    even_count = 0  
    odd_count = 0  
  
    while current:  
        if index % 2 == 0: # Even index (0, 2, 4, ...)  
            even_list.insert(current.data, even_count)  
            even_count += 1  
        else: # Odd index (1, 3, 5, ...)  
            odd_list.insert(current.data, odd_count)  
            odd_count += 1  
        current = current.next  
        index += 1  
  
    return even_list, odd_list
```

Q03 - Answer

Now we will go through the split() Function

- Create Two New Lists
- **even_list** → stores nodes from even positions.
- **odd_list** → stores nodes from odd positions.
- Both start as empty linked lists.



```
def split(ll):  
    if not ll.head:  
        return LinkedList(), LinkedList()  
  
    even_list = LinkedList()  
    odd_list = LinkedList()  
    current = ll.head  
    index = 0  
    even_count = 0  
    odd_count = 0  
  
    while current:  
        if index % 2 == 0: # Even index (0, 2, 4, ...)  
            even_list.insert(current.data, even_count)  
            even_count += 1  
        else: # Odd index (1, 3, 5, ...)  
            odd_list.insert(current.data, odd_count)  
            odd_count += 1  
        current = current.next  
        index += 1  
  
    return even_list, odd_list
```

Q03 - Answer

Now we will go through the split() Function

- **current** → pointer to traverse the original list.
- **index** → keeps track of the current node's position (0-based).
- **even_count** → how many elements have been inserted into even_list.
- **odd_count** → how many elements have been inserted into odd_list.
- These counts are needed because .insert(data, position) requires the position in the target list, not the original.



```
def split(ll):  
    if not ll.head:  
        return LinkedList(), LinkedList()  
  
    even_list = LinkedList()  
    odd_list = LinkedList()  
    current = ll.head  
    index = 0  
    even_count = 0  
    odd_count = 0  
  
    while current:  
        if index % 2 == 0: # Even index (0, 2, 4, ...)  
            even_list.insert(current.data, even_count)  
            even_count += 1  
        else: # Odd index (1, 3, 5, ...)  
            odd_list.insert(current.data, odd_count)  
            odd_count += 1  
        current = current.next  
        index += 1  
  
    return even_list, odd_list
```

Q03 - Answer

Now we will go through the split() Function

- Loops through the original linked list until current becomes None (end of list).



```
def split(ll):
    if not ll.head:
        return LinkedList(), LinkedList()

    even_list = LinkedList()
    odd_list = LinkedList()
    current = ll.head
    index = 0
    even_count = 0
    odd_count = 0

    while current:
        if index % 2 == 0: # Even index (0, 2, 4, ...)
            even_list.insert(current.data, even_count)
            even_count += 1
        else: # Odd index (1, 3, 5, ...)
            odd_list.insert(current.data, odd_count)
            odd_count += 1
        current = current.next
        index += 1

    return even_list, odd_list
```

Q03 - Answer

Now we will go through the split() Function

- If $\text{index} \% 2 == 0 \rightarrow$ position is even:
 - Insert the current node's data into **even_list** at position **even_count**.
 - Increase **even_count** because **even_list** got a new node.
- Otherwise (odd position):
 - Insert into **odd_list** at position **odd_count**.
 - Increase **odd_count**.



```
def split(ll):
    if not ll.head:
        return LinkedList(), LinkedList()

    even_list = LinkedList()
    odd_list = LinkedList()
    current = ll.head
    index = 0
    even_count = 0
    odd_count = 0

    while current:
        if index % 2 == 0: # Even index (0, 2, 4, ...)
            even_list.insert(current.data, even_count)
            even_count += 1
        else: # Odd index (1, 3, 5, ...)
            odd_list.insert(current.data, odd_count)
            odd_count += 1
        current = current.next
        index += 1

    return even_list, odd_list
```

Q03 - Answer

Now we will go through the split() Function

```
def split(ll):
    if not ll.head:
        return LinkedList(), LinkedList()

    even_list = LinkedList()
    odd_list = LinkedList()
    current = ll.head
    index = 0
    even_count = 0
    odd_count = 0

    while current:
        if index % 2 == 0: # Even index (0, 2, 4, ...)
            even_list.insert(current.data, even_count)
            even_count += 1
        else: # Odd index (1, 3, 5, ...)
            odd_list.insert(current.data, odd_count)
            odd_count += 1

        current = current.next
        index += 1

    return even_list, odd_list
```

- Moves the pointer to the next node in the original list.
- Increments **index** so the next iteration knows whether it's even or odd.



```
current = current.next
index += 1
```

Q03 - Answer

Now we will go through the split() Function

```
def split(ll):  
    if not ll.head:  
        return LinkedList(), LinkedList()  
  
    even_list = LinkedList()  
    odd_list = LinkedList()  
    current = ll.head  
    index = 0  
    even_count = 0  
    odd_count = 0  
  
    while current:  
        if index % 2 == 0: # Even index (0, 2, 4, ...)  
            even_list.insert(current.data, even_count)  
            even_count += 1  
        else: # Odd index (1, 3, 5, ...)  
            odd_list.insert(current.data, odd_count)  
            odd_count += 1  
        current = current.next  
        index += 1  
  
    return even_list, odd_list
```

- Returns two **completely separate linked lists**.
- Original list remains untouched.



4. Write a function **duplicateReverse()** that creates a duplicate of a linked list with the nodes stored in reverse.

The function prototype is given below:

def duplicateReverse(ll):

The function should return a new LinkedList object containing the reversed duplicate of the input linked list. If the input list is empty, return an empty LinkedList.

Q04 - Answer

Now we will go through the duplicateReverse() Function

- We start by making an empty linked list called **reversed_list**.
- This is where the reversed elements will be stored.



```
def duplicateReverse(l1):  
    # Create a new linked list for reversed copy  
    reversed_list = LinkedList()  
  
    if not l1.head:  
        return reversed_list  
  
    # Create new list in reverse order  
    current = l1.head  
    while current:  
        reversed_list.insert(current.data, 0)  
        current = current.next  
  
    return reversed_list
```

Q04 - Answer

Now we will go through the duplicateReverse() Function

- If the original list is empty (**ll.head** is **None**), we just return the empty **reversed_list**.
- This avoids unnecessary processing.



```
def duplicateReverse(ll):  
    # Create a new linked list for reversed copy  
    reversed_list = LinkedList()  
  
    if not ll.head:  
        return reversed_list  
  
    # Create new list in reverse order  
    current = ll.head  
    while current:  
        reversed_list.insert(current.data, 0)  
        current = current.next  
  
    return reversed_list
```

Q04 - Answer

Now we will go through the `duplicateReverse()` Function

- **current = ll.head** → start from the first node in the original list.
- The loop runs until **current** becomes **None** (end of list).



```
def duplicateReverse(ll):  
    # Create a new linked list for reversed copy  
    reversed_list = LinkedList()  
  
    if not ll.head:  
        return reversed_list  
  
    # Create new list in reverse order  
    current = ll.head  
    while current:  
        reversed_list.insert(current.data, 0)  
        current = current.next  
  
    return reversed_list
```

Q04 - Answer

Now we will go through the duplicateReverse() Function

- This is the trick that reverses the order:
 - Each new element from the original list is inserted at the beginning of **reversed_list**.
 - Inserting at **index 0** means the most recent item always becomes the new head.



```
def duplicateReverse(l1):  
    # Create a new linked list for reversed copy  
    reversed_list = LinkedList()  
  
    if not l1.head:  
        return reversed_list  
  
    # Create new list in reverse order  
    current = l1.head  
    while current:  
        reversed_list.insert(current.data, 0)  
        current = current.next  
  
    return reversed_list
```

Q04 - Answer

Now we will go through the `duplicateReverse()` Function

```
def duplicateReverse(ll):  
    # Create a new linked list for reversed copy  
    reversed_list = LinkedList()  
  
    if not ll.head:  
        return reversed_list  
  
    # Create new list in reverse order  
    current = ll.head  
    while current:  
        reversed_list.insert(current.data, 0)  
        current = current.next
```

- Finally, we return the new list, which is a reversed duplicate of the original.



```
return reversed_list
```